

Introduction

Every so often a LeetCode problem shows up wearing a disguise. On the surface, "Make Lexicographically Smallest Array by Swapping Elements" (LeetCode 2948) looks like a simulation problem — swap this pair, swap that pair, see what happens. Try to solve it that way and you'll burn through your interview time simulating swaps that never converge on the right answer.

This problem matters because it tests something interviewers care about far more than raw coding speed: can you recognize when a problem *isn't* the problem it appears to be? Underneath the swapping rules is a connectivity question, and the moment you see that, the solution almost writes itself. That reframing — from "simulate the operation" to "model the relationship" — is one of the most transferable skills in technical interviews, and it's exactly what Union-Find was built for.

Problem Overview

You're given an array of positive integers and a single integer `limit`. You may repeatedly swap any two elements in the array, but only if the absolute difference between their values is less than or equal to `limit`. You can perform this swap as many times as you like, in any order.

Your task is to produce the lexicographically smallest version of the array reachable through any sequence of these swaps. Lexicographically smallest means: compare the two candidate arrays left to right, and whichever one has the smaller value at the first position where they differ wins.

The catch is scale. The array can hold up to 100,000 elements, and both the values and the limit can be as large as a billion. Any approach that checks every pair of positions, or that tries to bucket values by counting, is off the table. You need something close to sorting speed.

Example

Take the array `[1, 5, 3, 9, 8]` with `limit = 2`.

- Swap positions 1 and 2 (values 5 and 3, difference 2 — legal): `[1, 3, 5, 9, 8]`
- Swap positions 3 and 4 (values 9 and 8, difference 1 — legal): `[1, 3, 5, 8, 9]`

No further legal swap improves this. The final answer is `[1, 3, 5, 8, 9]`.

Notice something curious: 1 never moves, but 9 and 8 do. And even though 1 and 9 differ by 8 — far outside the limit — you might see problems where a value ends up sitting right where a very different value used to be, because chains of small differences add up. That's the phenomenon this problem is really testing.

Intuition

The first instinct almost everyone has is to treat this literally: find any two positions whose values are close enough, swap them, check if the array improved, repeat. That works for the easy cases, but it breaks down fast. A value can be "stuck" relative to any single swap and still need to travel far across the array — it just has to travel through a chain of smaller hops.

Here's the reframe: forget about swaps as actions. Think about them as *edges*. If value A can swap with value B, draw a line between them. If B can swap with C, draw another line. Even if A and C can never swap directly, they're connected through B — and because swaps can happen any number of times in any order, anything connected through a chain of legal swaps can eventually end up anywhere else in that chain's positions.

That's a graph connectivity problem, and the tool for graph connectivity at this scale, without building an actual graph, is Union-Find (also called Disjoint Set Union). Once you see "chains of connections," union-find should come to mind almost as a reflex.

The remaining trick is *how* you build those connections. If you compare adjacent elements in the original array, you're really asking "are these two positions currently next to each other," which has nothing to do with whether they can swap — swapping depends on value difference, not position. Instead, sort the values first. Once sorted, any two neighboring values that differ by no more than `limit` are legally linked, and because they're sorted, this single adjacent-pair scan is guaranteed to catch every possible link — a value further away in the sorted order can only be connected through a chain of these adjacent links anyway.

Brute Force Solution

Idea: Repeatedly scan for any pair of positions whose values differ by at most `limit`, swap them if it produces a smaller array, and repeat until no improving swap exists.

Algorithm: 1. Loop over all pairs of positions. 2. For each pair, check if the value difference is within `limit`. 3. If swapping would decrease the array lexicographically, perform it. 4. Repeat until a full pass produces no improvement.

Advantages: Simple to reason about and implement for small inputs; directly mirrors the problem statement.

Disadvantages: Doesn't reliably reach the true optimum, because a value might need several swaps in sequence — including some that look locally "non-improving" — to reach its ideal final position. It also doesn't terminate cleanly at a known optimal state without extra bookkeeping.

Time complexity: $O(n^2)$ per pass, with potentially many passes needed — easily $O(n^3)$ or worse in practice. **Space complexity:** $O(1)$ beyond the array itself.

For n up to 100,000, this is many orders of magnitude too slow.

Optimal Solution

The optimal approach has four moving parts:

1. **Sort indices by value.** We need to know both the sorted order of values and which original position each value came from, so we sort a list of indices using the array values as the sort key — not the values themselves.
2. **Build clusters with Union-Find.** Walk through the sorted order. For each pair of neighbors, if their values differ by at most `limit`, union their *original* positions. Because the values are sorted, this catches every reachable link with a single linear scan — no need to compare every pair.
3. **Group positions by cluster.** After all unions are done, every position belongs to exactly one root. Group all positions sharing a root into a cluster.
4. **Refill each cluster optimally.** For every cluster, take its set of original positions, sort them, take its set of values, sort them, and assign the smallest value to the earliest position, the next smallest to the next position, and so on.

Why does this produce the lexicographically smallest array? Because a cluster represents every position a value inside it could ever reach through some sequence of legal swaps — and nothing outside the cluster can ever mix in. Given a fixed set of positions and a fixed set of values that must occupy exactly those positions, the lexicographically smallest arrangement is always "smallest value goes in the earliest position." There's no cleverer arrangement possible once positions and values are locked to a cluster.

Sorted value Original index Linked to next? ($\leq$ limit)

1	0	yes (diff 2)
3	2	yes (diff 2)

Sorted value Original index Linked to next? ($\leq$ limit)

5	1	no (diff 3)
8	4	yes (diff 1)
9	3	—

This gives two clusters: positions $\{0, 2, 1\}$ with values $\{1, 3, 5\}$, and positions $\{4, 3\}$ with values $\{8, 9\}$. Sorting each cluster's positions and values and matching them up gives exactly $[1, 3, 5, 8, 9]$.

Python Code

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path compression
            x = self.parent[x]
        return x

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)
        if root_x != root_y:
            self.parent[root_x] = root_y

def lexicographically_smallest_array(nums, limit):
    n = len(nums)
    uf = UnionFind(n)

    # Sort original indices by their value
    order = sorted(range(n), key=lambda i: nums[i])

    # Union adjacent indices (in sorted-value order) that are swappable
    for i in range(n - 1):
        if nums[order[i + 1]] - nums[order[i]] <= limit:
            uf.union(order[i], order[i + 1])

    # Group original positions by their cluster root
    clusters = {}
    for i in range(n):
        root = uf.find(i)
        clusters.setdefault(root, []).append(i)

    result = nums[:]
    for positions in clusters.values():
        positions.sort()
```

```

        values = sorted(nums[i] for i in positions)
        for pos, val in zip(positions, values):
            result[pos] = val

    return result

```

Code Walkthrough

- `UnionFind.__init__` gives every position its own parent — itself — so every element starts in its own singleton cluster.
- `find` walks up the parent chain until it hits a self-parenting node (the root), and along the way it repoints each node to its grandparent, flattening the structure so future lookups stay fast — this is path compression.
- `union` finds the roots of both inputs and, if they differ, merges the clusters by pointing one root at the other.
- `order = sorted(range(n), key=lambda i: nums[i])` produces the indices of `nums` sorted by their values, without discarding position information.
- The union loop compares each pair of *sorted-order neighbors*; if they're within `limit`, their original indices get unioned. This is the step that correctly encodes "can swap" as "value difference," not position.
- The clustering dictionary groups every position under its cluster's root.
- The final loop takes each cluster's positions and values, sorts both independently, and pairs them smallest-to-smallest — this placement step is what actually produces the answer.

Dry Run

Using `nums = [1, 5, 3, 9, 8]`, `limit = 2`:

1. `order` sorted by value: indices `[0, 2, 1, 4, 3]` (values `1, 3, 5, 8, 9`).
2. Union pass: `1-3` diff 2 → `union(0, 2)`. `3-5` diff 2 → `union(2, 1)`. `5-8` diff 3 → skip. `8-9` diff 1 → `union(4, 3)`.
3. Clusters: `{0, 1, 2}` and `{3, 4}`.
4. Cluster `{0, 1, 2}`: positions sorted `[0, 1, 2]`, values sorted `[1, 3, 5]` → `result[0]=1`, `result[1]=3`, `result[2]=5`.
5. Cluster `{3, 4}`: positions sorted `[3, 4]`, values sorted `[8, 9]` → `result[3]=8`, `result[4]=9`.
6. Final result: `[1, 3, 5, 8, 9]`. Matches the expected answer.

Complexity Analysis

Time: $O(n \log n)$, dominated by the initial sort of indices by value. The union-find operations, with path compression, run in effectively $O(n \alpha(n))$ — nearly linear, where α is the inverse Ackermann function. Grouping and re-sorting each cluster costs no more than $O(n \log n)$ total across all clusters, since the sum of cluster sizes is n .

Space: $O(n)$ for the parent array, the sorted-order list, and the cluster dictionary.

This is correct because the only operation that costs more than linear time is sorting, and it happens a bounded number of times (once for the full array, once per cluster, but the cluster sorts together never exceed n elements total).

Alternative Solutions

You could model this as an explicit graph and run BFS or DFS to find connected components instead of Union-Find. It produces the same clusters, but requires building an adjacency structure and generally runs slower in practice due to higher constant factors and stack/queue overhead — Union-Find with path compression is the tighter fit here because the connections are formed incrementally, in sorted order, rather than needing to be looked up as a real graph.

Edge Cases

- **Empty or single-element array:** No swaps possible or needed; return as-is.
- **All values identical:** Every element is within `limit` of every other; the whole array collapses into a single cluster, and since all values are equal, the "smallest arrangement" is just the original array.
- **`limit = 0`:** No two distinct values can ever swap (since a difference of 0 means values are already equal); duplicates of the same value can still be considered connected, but sorting them changes nothing.
- **Very large values near the billion mark:** Ensure difference calculations don't rely on any fixed-width overflow assumptions (not an issue in Python, but worth calling out for other languages).
- **Array already sorted:** Every cluster's values are already in position; the algorithm still runs but performs no meaningful reordering.

Common Mistakes

1. **Unioning adjacent original indices** (`i` and `i+1`) instead of adjacent *sorted-value* indices — this conflates position with value proximity.
2. **Forgetting path compression**, which turns `find` into a potential $O(n)$ operation per call and blows up the time complexity.
3. **Sorting values without preserving original indices**, losing the ability to write results back to the correct positions.
4. **Assuming a single swap must directly connect two far-apart values**, missing that indirect chains of swaps allow far-apart values to reach each other.
5. **Re-sorting the entire array instead of per-cluster**, which ignores the constraint that a value can only move within its own connected cluster.

Interview Questions

- How would you prove that sorting each cluster's values into its sorted positions is optimal?
- What changes if swaps have a cost, and you want the smallest array within a budget of swaps?
- How would you extend this to report the minimum number of actual swaps needed to reach the final array?
- What if `limit` could be different for every pair of indices instead of a single global value?

Similar Problems

- **LeetCode 947 (Most Stones Removed with Same Row or Column)**: Another connectivity problem solved cleanly with Union-Find.
- **LeetCode 1319 (Number of Operations to Make Network Connected)**: Uses Union-Find to count connected components directly.
- **LeetCode 721 (Accounts Merge)**: Union-Find over a different kind of "closeness" relation — shared emails instead of value difference.
- **LeetCode 990 (Satisfiability of Equality Equations)**: Union-Find applied to equivalence constraints rather than numeric proximity.

Key Takeaways

The core lesson of LeetCode 2948 is recognizing when an operation-based problem ("you can swap these") is secretly a relationship-based problem ("these things are connected"). Once you sort the values and treat "swappable" as an edge, Union-Find turns a seemingly complex simulation into a clean, near-linear algorithm. The two ideas to hold onto: connect by value proximity, not position, and once positions and values are locked into a cluster, sorted-to-sorted placement is always optimal.

Watch the Video

For the full walkthrough — including a live timing comparison between the brute-force and optimal approaches, and two quiz checkpoints to test your understanding — watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series dedicated to building real intuition behind coding interview problems rather than memorizing solutions. Each episode traces a problem from first read to optimal solution, with runnable code and complexity proof along the way.

Call To Action

If this helped you understand Union-Find a little better, give the video a like on YouTube and subscribe for more problem breakdowns like this one. For deeper dives and behind-the-scenes notes, subscribe to the Substack newsletter. Drop a comment with your own approach or a problem you'd like covered next, and share this with a friend who's prepping for interviews.

Quick Review

Pattern for 'swap only where $\text{diff} \leq \text{limit}$ ' problems?

Union-Find grouping: elements connected by allowed swaps form a group; sort within each group freely.

Time complexity of LC 2948 solution?

$O(n \log n)$ — dominated by sorting; union-find operations are near $O(n)$ with path compression.

Space complexity of LC 2948 solution?

$O(n)$ for union-find parent array, index groups, and sorted value groups.

Common bug when solving LC 2948?

Forgetting swaps are transitive — if a-b and b-c are swappable, a-c must join the same group even without a direct edge.

Why doesn't greedy pairwise swapping work here?

A single swap can only fix one pair; the real reachable set per index is its whole union-find component, not just direct neighbors.

Union-Find vs adjacency-list BFS for grouping — why prefer Union-Find?

Both find connected components, but union-find avoids building/traversing a graph explicitly, giving near-linear time with simpler code.

Make Lexicographically Smallest Array by Swapping Elements — free Python cheat sheet